

14.7 Transactional Data Architecture

Overview

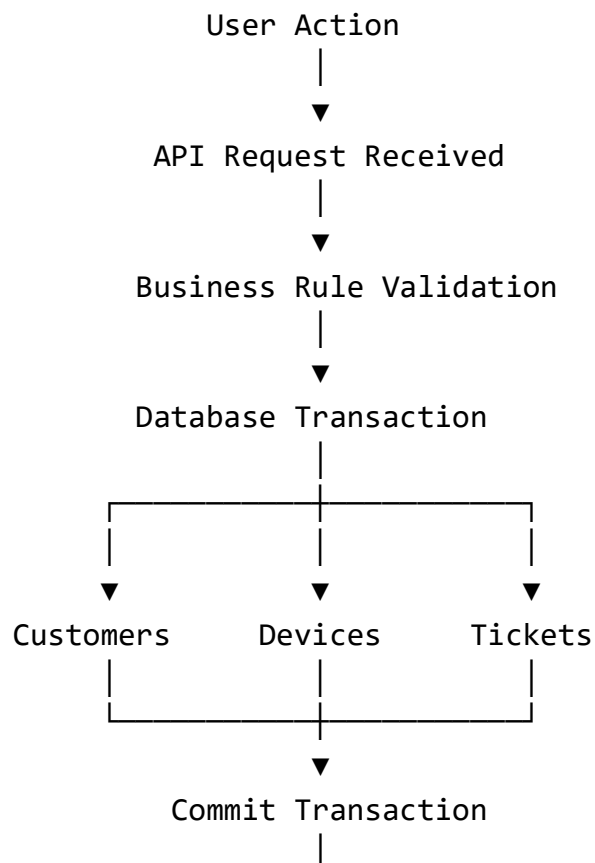
The Transactional Data Architecture defines how operational business data is created, updated, and maintained during the execution of platform services.

The architecture ensures that every business transaction is:

- Atomic
- Consistent
- Isolated
- Durable (ACID)

This guarantees that customer operations remain reliable even under high concurrency.

Transaction Flow



▼
Response Returned

Transaction Boundaries

A business transaction may involve multiple entities but must behave as a single logical unit.

Examples:

Customer Registration

Creates:

- Customer
- Customer Profile
- Communication Preferences
- Initial Conversation Context

If any operation fails, the entire transaction is rolled back.

Ticket Creation

Creates:

- Ticket
- Initial Status
- Conversation Record
- AI Analysis
- Audit Event

The ticket is considered valid only when all associated records are successfully committed.

ACID Compliance

The transactional layer follows the ACID model:

Atomicity

All operations succeed or none are persisted.

Consistency

Every transaction preserves database integrity.

Isolation

Concurrent transactions do not interfere with each other.

Durability

Committed transactions survive system failures.

Transaction Types

The platform distinguishes between:

Read Transaction

Write Transaction

Batch Transaction

AI-Assisted Transaction

Administrative Transaction

Background Transaction

Each type follows its own optimization and validation strategy.

Concurrency Control

To maintain consistency under concurrent access, the platform supports:

- Optimistic locking where appropriate.
- Database constraints for integrity.
- Transaction isolation levels provided by PostgreSQL.
- Idempotent operations for retryable requests.

Event Generation

Successful transactions may emit domain events, such as:

CustomerCreated

DeviceRegistered

TicketOpened

TicketClosed

KnowledgeUpdated

ConversationStarted

These events can later be consumed by analytics, notifications, integrations, or future event-driven services without coupling them to the transactional workflow.